```
    hx-swap="outerHTML">
    <i> Loading... </i>
  </div>
{% endif %}
```

Display a Loading… message until the results are available.

A value of "outerHTML" means to replace the entire with the results from the GET call.

This new addition to the results partial implements infinite scrolling. It is common to indicate that something is loading if it takes a bit of time; I've used text here, but you could use an animated spinner GIF instead. Since the search view has a delay in it, you should see the Loading… text on the screen for 2 seconds before more results get shown.

You could test this out now, but as you don't have many musicians, there won't be enough data to trigger the infinite scroll feature. That is easily fixed. Invoke your Django shell with the `shell` management command, and enter the following code:

```
>>> from datetime import date
>>> from bands.models import Musician
>>> for n in range(1, 81):
...     Musician.objects.create(first_name=f"X{n:02}", last_name="Z",
...     ➥ birth=date(1900, 1, 20))
...
```

You now have an extra 80 musicians, all of whom conveniently have a first name starting with the same letter. Restart your development server, and try out your search results page with infinite scroll. Don't forget to keep scrolling until the data is consumed; you want to make sure the last page is working as well.

12.5 Click to edit

The Django Admin is very Web 1.0. It starts with a list of items, and to edit one, you click it and go to the next screen. With HTMX, you can perform inline editing, modifying the page to replace an item with the form to edit it. This is known as *click to edit*. The concepts are similar to what you've seen already: `hx-get` to replace a tag with a new partial, where the partial contains the edit form. HTMX supports all the HTTP methods you're familiar with, so as you might guess, the form gets submitted with `hx-post`.

To demonstrate click-to-edit, let's build an editor for the rooms associated with a venue. For this feature, you'll need three views: the main page displaying a venue's rooms, a view that handles the room-editing form, and a third view that can display a room partial on its own. The third view is necessary to give the user the ability to cancel out of the edit.

The main page view for editing rooms takes the ID of a `Venue` object and displays its associated rooms. The view is about as simple as is possible. The following listing shows the view I've added to RiffMates/bands/views.py.

Listing 12.12 A view to display the rooms in a venue

```
# RiffMates/bands/views.py
...
from django.contrib.auth.decorators import login_required
from django.shortcuts import render, get_object_or_404
```